NAIJA NEST — SYSTEM ARCHITECTURE & TECHNICAL STRATEGY

Backend Architecture Design Document Version 1.0

Confidential — Engineering Internal Use Only

Document Owner: Backend Architecture & Lead Engineering

SECTION 1: HIGH-LEVEL SYSTEM ARCHITECTURE

1.1 Architectural Pattern Decision: Modular Monolith → Microservices Migration Path

Recommendation: Modular Monolith with Bounded Context Isolation (Phase 1), Designed for Microservices Extraction (Phase 2+)

For a platform at NaijaNest's current stage, pre-product-market-fit, small engineering team, rapidly evolving domain model, a pure microservices architecture would introduce unnecessary operational complexity, distributed tracing overhead, and inter-service latency that the team is not yet positioned to manage effectively.

Instead, we adopt a Modular Monolith where each bounded domain (Users/KYC, Properties, Agents, Verification, AI/NLP, Notifications) is a clearly isolated module with strict intra-module API contracts and zero cross-module direct database access. Each module is a candidate for extraction into an independent microservice when load and team size justify it, without requiring a rewrite.

The one critical exception is the AI/ML processing pipeline. This is extracted as an independent service from Day 1, for two reasons: it has fundamentally different scaling characteristics (GPU/compute-intensive, bursty workloads) and it is likely written in Python (where the ML ecosystem lives), making it a natural polyglot service boundary.

Architectural Tiers

Tier 1 — API Gateway Layer A single entry point for all mobile and web clients. Handles authentication token validation, rate limiting, request routing, and SSL termination. No business logic lives here.

Tier 2 — Core Application Monolith (Spring Boot) Contains all bounded domain modules. Modules communicate via internal Spring application events (not HTTP) within the monolith. Each module owns its own schema namespace within the shared relational database. This maintains data consistency while preserving logical separation.

Tier 3 — AI/ML Processing Service (Python FastAPI) An independent polyglot service responsible for document forensic analysis, NLP embeddings generation, LLM conversation orchestration, and sentiment analysis. Receives work via message queue, not direct HTTP calls.

Tier 4 — Async Event Infrastructure (Apache Kafka) All trust framework operations — document verification, fraud detection, trust score recalculation, notification dispatch — are event-driven and asynchronous. No user-facing request blocks on these operations.

Tier 5 — Data Layer Polyglot persistence: PostgreSQL for transactional data, MongoDB for property listings and chat history, Elasticsearch for property search, Weaviate (vector database) for AI embeddings, Redis for caching and session management, and AWS S3 (or Cloudflare R2) for secure document and media storage.

1.2 High-Level Architecture Diagram (Mermaid.js)

```
graph TB
  subgraph CLIENT_LAYER["Client Layer"]
    MOB[Mobile App Android/iOS]
    WEB[Web App React]
    DIASPORA[Diaspora Remote Client]
  end

  subgraph GATEWAY["API Gateway Layer - Kong / AWS API Gateway"]
    GW[API Gateway]
    AUTH_MW[Auth Middleware JWT/OAuth2]
    RATE[Rate Limiter]
    LB[Load Balancer]
  end

  subgraph CORE["Core Application - Spring Boot Modular Monolith"]
    direction TB
    USER_MOD[User & KYC Module]
    PROP_MOD[Property Module]
    AGENT_MOD[Agent Module]
    VERIF_MOD[Verification Orchestration Module]
    TRUST_MOD[Trust Score Module]
    SEARCH_MOD[Search & Recommendation Module]
    NOTIF_MOD[Notification Module]
    CHAT_MOD[AI Chat Orchestration Module]
  end
```

```
subgraph EVENTS["Event Bus - Apache Kafka"]
   K1[document.submitted]
   K2[fraud.signal.detected]
   K3[trust.score.recalculate]
   K4[notification.dispatch]
   K5[listing.verified]
   K6[viewing.completed]
end

subgraph AI_SERVICE["AI/ML Processing Service - Python FastAPI"]
   DOC_VERIF[Document Forensic Analyser]
   NLP_ENGINE[NLP Search Intent Parser]
   SENT_ANAL[Sentiment Analyser]
   LLM_ORCH[LLM Conversation Orchestrator]
   EMBED_GEN[Embedding Generator]
end

subgraph DATA_LAYER["Data Layer"]
   PG[(PostgreSQL - Transactional)]
   MONGO[(MongoDB - Listings & Chat)]
   ES[(Elasticsearch - Search Index)]
   WEAVIATE[(Weaviate - Vector DB)]
   REDIS[(Redis - Cache & Sessions)]
   S3[(Object Storage - Documents/Media)]
end

subgraph EXTERNAL["External Integrations"]
   BVN[NIBSS BVN/NIN API]
   MAPS[Google Maps Geocoding]
   LLM_EXT[OpenAI / Claude API]
   SMS[Termii / Africa's Talking SMS]
   PUSH[Firebase FCM Push]
end

MOB --> GW
WEB --> GW
DIASPORA --> GW
GW --> AUTH_MW --> RATE --> LB --> CORE

CORE --> EVENTS
```

```
EVENTS --> AI_SERVICE
EVENTS --> NOTIF_MOD

AI_SERVICE --> WEAVIATE
AI_SERVICE --> LLM_EXT

CORE --> PG
CORE --> MONGO
CORE --> REDIS
CORE --> ES
SEARCH_MOD --> ES
SEARCH_MOD --> WEAVIATE

VERIF_MOD --> S3
AI_SERVICE --> S3

USER_MOD --> BVN
PROP_MOD --> MAPS
NOTIF_MOD --> SMS
NOTIF_MOD --> PUSH
```

1.3 Event-Driven Architecture for the Trust Framework

The Trust Framework is the most critical and complex subsystem in NaijaNest. Every operation within it is asynchronous and event-sourced. No user-facing API call blocks on a verification task.

Trust Framework Event Flow

When an agent submits a property listing with documents, the following event chain is triggered:

Step 1 — Document Submitted Event The Verification Orchestration Module publishes a document.submitted event to Kafka. The event payload includes: listing ID, agent ID, document type, and a pre-signed S3 URL pointing to the uploaded file. The API returns a 202 Accepted response to the agent immediately — the listing enters "Pending Verification" status.

Step 2 — AI Forensic Analysis The AI/ML Processing Service consumes the document.submitted event. It retrieves the document from S3, runs forensic analysis (metadata, image manipulation detection, OCR extraction), and publishes a verification.task.completed event with a risk score and flagged anomalies.

Step 3 — Trust Score Recalculation The verification.task.completed event triggers the Trust Score Module to recalculate the agent's composite Trust Score based on the new data point.

Step 4 — Fraud Signal Propagation If the risk score exceeds the fraud threshold, a fraud.signal.detected event is published. This triggers: automatic listing suspension, notification to the Trust & Safety team queue, and if the agent has a pattern of fraud signals, automatic account suspension.

Step 5 — Notification Dispatch A notification.dispatch event is published regardless of outcome — to notify the agent of the verification result and, on success, to move the listing to "Verified" status and update the search index.

Trust Framework Event Flow Diagram (Mermaid.js)

```
sequenceDiagram
  participant Agent as Agent (Mobile)
  participant API as API Gateway
  participant VERIF as Verification Module
  participant KAFKA as Kafka Event Bus
  participant AI as AI/ML Service
  participant S3 as Object Storage
  participant TRUST as Trust Score Module
  participant SAFETY as Trust & Safety Queue
  participant NOTIF as Notification Service

  Agent->>API: POST /listings with documents
  API->>VERIF: Route request
  VERIF->>S3: Upload documents (server-side encrypted)
  VERIF->>KAFKA: Publish document.submitted event
  VERIF->>API: Return 202 Accepted (listing = PENDING)
  API->>Agent: 202 Accepted - Verification in progress

  KAFKA->>AI: Consume document.submitted
  AI->>S3: Retrieve document via pre-signed URL
  AI->>AI: Run forensic analysis pipeline
  AI->>KAFKA: Publish verification.task.completed (risk_score, flags)

  KAFKA->>VERIF: Consume verification.task.completed
  VERIF->>KAFKA: Publish trust.score.recalculate

  alt Risk Score ABOVE Threshold
    KAFKA->>TRUST: Recalculate score with fraud signal
```

```
    VERIF->>KAFKA: Publish fraud.signal.detected
    KAFKA->>SAFETY: Route to human review queue
    KAFKA->>NOTIF: Publish notification.dispatch (SUSPENSION)
    NOTIF->>Agent: Push/SMS - Listing suspended, review pending
  else Risk Score BELOW Threshold
    KAFKA->>TRUST: Recalculate score with positive signal
    VERIF->>KAFKA: Publish listing.verified
    KAFKA->>NOTIF: Publish notification.dispatch (APPROVED)
    NOTIF->>Agent: Push/SMS - Listing verified and live
  end
```

SECTION 2: TECHNOLOGY STACK (JAVA ECOSYSTEM)

2.1 Framework Decision: Spring Boot 3.x with Spring Modulith

Primary Framework: Spring Boot 3.x (Java 21)

Spring Boot remains the most appropriate choice for this platform for the following reasons:

Spring Boot's mature ecosystem directly covers every NaijaNest requirement out of the box — Spring Security for authentication and authorisation, Spring Data JPA for PostgreSQL, Spring Data MongoDB for NoSQL persistence, Spring Kafka for event-driven messaging, and Spring AI (emerging) for LLM integration. The talent pool for Spring Boot in Nigeria is significantly larger than alternatives like Micronaut or Quarkus, which is critical for team scaling. Java 21 brings Virtual Threads (Project Loom), which dramatically improves the platform's ability to handle high I/O concurrency (e.g., simultaneous BVN/NIN API calls) without the complexity of reactive programming.

Modularity Extension: Spring Modulith

Spring Modulith enforces the bounded context isolation of the Modular Monolith pattern. It provides compile-time enforcement of module boundaries (preventing circular dependencies), built-in event publication with transactional guarantees (events are only published if the originating transaction commits), and automatic generation of module documentation. This is the architectural guardrail that makes the monolith-to-microservices migration path clean.

Build Tool: Gradle (Kotlin DSL)

Gradle over Maven for its superior incremental build performance, more expressive build scripts, and better support for multi-module project structures.

API Design: REST with OpenAPI 3.1

All external APIs are RESTful and documented via SpringDoc OpenAPI. Internal module-to-module communication within the monolith uses Spring Application Events exclusively — never direct method calls across module boundaries.

2.2 Database Strategy: Polyglot Persistence

PostgreSQL 15 — Transactional Core

Purpose: All data requiring ACID guarantees — user accounts, KYC records, agent profiles, verification task state, viewing appointments, financial transaction records, audit logs, and trust score history.

Rationale: The Trust Framework is fundamentally a state machine with strict consistency requirements. A document verification task must never be in an ambiguous state. A trust score must be calculated on a consistent, auditable record. Eventual consistency is not acceptable for these domains. PostgreSQL's row-level security (RLS) provides a native mechanism for enforcing data access policies at the database layer, providing a critical second line of defence for PII protection.

Key Design Decisions:

- Separate schema namespaces per bounded domain (schema: user_kyc, agent, property_core, verification, trust)
- All tables have created_at, updated_at, created_by audit columns enforced at the database level via triggers
- Soft deletes only — no hard deletes for any record in the verification or user_kyc schemas (compliance and audit requirement)

MongoDB 7.0 — Property Listings & Chat History

Purpose: Property listing documents (rich, schema-flexible, nested data), AI conversation history, neighbourhood data snapshots, and user shortlist/preference data.

Rationale: Property listings are inherently schema-flexible — a Lagos flat and an Abuja detached house have fundamentally different attributes. Forcing this into a rigid relational schema creates either a massive, sparse table or an EAV (Entity-Attribute-Value) anti-pattern. MongoDB's document model maps naturally to property listing data and allows seamless addition of new attributes (e.g., a "solar power rating" field added for V2) without schema migrations. Chat history is a natural document store use case — each conversation session is a self-contained document with an array of message objects.

Key Design Decisions:

- Property listing documents embed neighbourhood data as a nested sub-document (denormalised for read performance)
- Conversation history collections have a TTL index — inactive conversations older than 90 days are automatically purged
- Change Streams on the properties collection feed real-time updates to the Elasticsearch index

Elasticsearch 8.x — Search & Indexing

Purpose: Full-text search across property listings, agent profiles, and neighbourhood descriptions. Geo-spatial queries (properties within X km of a given location). Faceted filtering (price range, bedroom count, verification tier). Query suggestions and autocomplete.

Rationale: Property search is the platform's primary user interaction. PostgreSQL full-text search is insufficient for the combination of semantic text matching, faceted filtering, geo-distance queries, and the result scoring flexibility that a property search requires. Elasticsearch is the industry standard for this use case.

Key Design Decisions:

- Properties are indexed in Elasticsearch as a denormalised document (listing data + agent summary + verification badge + neighbourhood score). This is a read replica — the source of truth remains MongoDB.
- A Kafka Consumer in the Search Module listens for listing.verified and listing.updated events and incrementally updates the Elasticsearch index. Full re-indexing is a scheduled nightly job.
- Geo-point fields on all listings enable native distance-based sorting and bounding box filtering.

Weaviate — Vector Database for AI Embeddings

Purpose: Storing and querying vector embeddings of property listings, neighbourhood descriptions, and user preference profiles for semantic search and AI-powered recommendation.

Rationale: The "AI Conversational Property Search" (PRD Feature M1) requires understanding semantic intent, not just keyword matching. When a user says "I want somewhere quiet with good schools and not too far from the expressway," no keyword search can resolve this. The NLP pipeline generates a vector embedding of the user's intent, and Weaviate performs a nearest-neighbour search across listing embeddings to find the most semantically similar properties. This is what makes the AI search feel genuinely intelligent rather than a filtered keyword search.

Key Design Decisions:

- Each property listing's description, AI-generated summary, and neighbourhood data are embedded and stored in Weaviate.
- User preference profiles (built from search history and shortlist behaviour) are also embedded, enabling progressive personalisation.
- Weaviate's hybrid search mode (vector + keyword BM25) is used for all property queries to balance semantic relevance with exact attribute matching.

Redis 7.x — Caching, Sessions & Rate Limiting

Purpose: API response caching (neighbourhood data, agent profiles), user session storage, JWT token blacklisting, real-time rate limiting counters, and distributed locks for critical operations (e.g., preventing duplicate viewing bookings).

2.3 Search & Indexing: Conversational Property Search Architecture

The Conversational Property Search is a two-stage pipeline:

Stage 1 — NLP Intent Parsing (AI Service) The user's natural language query is sent to the AI/ML Service. The LLM extracts structured intent: city, area, property type, bedroom count, budget range, and any special requirements (schools, power supply, proximity constraints). This produces a structured SearchIntent JSON object.

Stage 2 — Hybrid Search Execution (Search Module) The SearchIntent object drives a parallel dual-query strategy. A structured Elasticsearch query handles the hard filters (city, price range, bedroom count, verification tier). Simultaneously, the query text is embedded and a Weaviate vector search identifies semantically relevant listings that match the soft requirements (lifestyle, proximity, amenity preferences). The results from both queries are merged and re-ranked by a composite score that weights: verification tier (highest weight), vector semantic similarity, price-to-budget ratio, agent trust score, and recency. The final ranked list is returned to the user.

2.4 Messaging & Queues: Apache Kafka

Why Kafka over RabbitMQ or SQS?

NaijaNest's Trust Framework requires event replay capability — if the AI service is temporarily unavailable, submitted documents must not be lost and must be processable when the service recovers. Kafka's durable, log-based storage with configurable retention (7 days default)

provides this. Kafka also supports multiple independent consumer groups reading the same event, which is critical: a single document.submitted event needs to be consumed by both the AI Service (for forensic analysis) and the Audit Log Service (for compliance recording) independently.

Kafka Topic Design:

Each topic follows the naming convention: [domain].[entity].[event]

- verification.document.submitted — New document received for analysis
- verification.task.completed — AI analysis result ready
- trust.score.recalculate — Trigger trust score recalculation for an agent
- fraud.signal.detected — Fraud pattern identified requiring human review
- listing.status.verified — Listing cleared, trigger search index update
- listing.status.suspended — Listing suspended, trigger removal from index
- notification.user.dispatch — Outbound notification to user
- notification.agent.dispatch — Outbound notification to agent
- viewing.appointment.confirmed — Viewing booked, trigger reminder scheduling
- viewing.appointment.completed — Viewing occurred, trigger review prompt

Consumer Group Strategy:

Topic verification.document.submitted has two consumer groups: ai-forensic-analysis (AI Service) and audit-compliance-logger (Audit Module within the monolith). Both receive every event independently. This decouples compliance recording from AI processing.

SECTION 3: DATA DESIGN

3.1 Core Entity-Relationship Model Concept

The data model is organised across 5 bounded domain schemas. Cross-domain references use foreign key IDs only — no cross-schema JOINs in application queries (this would violate module boundaries). Denormalised read models in Elasticsearch and MongoDB handle query requirements that would otherwise require cross-domain JOINs.

Domain: user_kyc Schema (PostgreSQL)

users table

- id (UUID, PK)
- phone_number (VARCHAR, UNIQUE, NOT NULL) — primary identifier
- email (VARCHAR, UNIQUE, NULLABLE)
- password_hash (VARCHAR, NOT NULL)

- role (ENUM: RENTER, BUYER, AGENT, ADMIN, TRUST_SAFETY)
- account_status (ENUM: PENDING_KYC, ACTIVE, SUSPENDED, CLOSED)
- created_at, updated_at, last_login_at (TIMESTAMPTZ)

kyc_profiles table

- id (UUID, PK)
- user_id (UUID, FK → users.id, UNIQUE)
- nin_hash (VARCHAR) — stored as SHA-256 hash, never plaintext
- bvn_hash (VARCHAR) — stored as SHA-256 hash, never plaintext
- nin_verification_status (ENUM: NOT_SUBMITTED, PENDING, VERIFIED, FAILED)
- bvn_verification_status (ENUM: NOT_SUBMITTED, PENDING, VERIFIED, FAILED)
- nin_verified_at, bvn_verified_at (TIMESTAMPTZ)
- verification_provider (VARCHAR) — e.g., "NIBSS_DIRECT", "DOJAH", "SMILE_IDENTITY"
- pii_vault_reference (VARCHAR) — reference token to PII Vault (see Section 5.2)

kyc_audit_log table

- id (UUID, PK)
- user_id (UUID, FK)
- event_type (ENUM: KYC_INITIATED, KYC_VERIFIED, KYC_FAILED, DATA_ACCESSED, DATA_EXPORTED)
- event_data (JSONB) — context of the event, sanitised
- performed_by (UUID) — user or system actor
- ip_address (INET)
- created_at (TIMESTAMPTZ)

Domain: agent Schema (PostgreSQL)

agents table

- id (UUID, PK)
- user_id (UUID, FK → user_kyc.users.id, UNIQUE)
- agency_name (VARCHAR, NULLABLE)
- cac_registration_number (VARCHAR, UNIQUE, NULLABLE)
- lasrera_licence_number (VARCHAR, UNIQUE, NULLABLE)
- agent_type (ENUM: INDIVIDUAL, AGENCY)
- cac_verification_status (ENUM: NOT_SUBMITTED, PENDING, VERIFIED, FAILED)
- lasrera_verification_status (ENUM: NOT_APPLICABLE, PENDING, VERIFIED, FAILED)

- active_city (ENUM: LAGOS, ABUJA, PORT_HARCOURT, OWERRI)
- subscription_tier (ENUM: FREE, STANDARD, PREMIUM)
- created_at, updated_at

agent_trust_scores table

- id (UUID, PK)
- agent_id (UUID, FK → agents.id, UNIQUE)
- composite_score (DECIMAL 3,1) — 0.0 to 10.0
- listing_accuracy_score (DECIMAL 3,1) — derived from post-viewing feedback
- transaction_completion_score (DECIMAL 3,1) — completed vs ghost viewings
- document_quality_score (DECIMAL 3,1) — from AI verification history
- review_sentiment_score (DECIMAL 3,1) — from NLP sentiment analysis
- fraud_report_penalty (DECIMAL 3,1) — negative score from fraud reports
- total_verified_listings (INT)
- total_completed_transactions (INT)
- total_fraud_reports (INT)
- last_calculated_at (TIMESTAMPTZ)

agent_trust_score_history table

- id (UUID, PK)
- agent_id (UUID, FK)
- score_snapshot (DECIMAL 3,1)
- change_trigger (ENUM: NEW_VERIFICATION, REVIEW_SUBMITTED, FRAUD_REPORT, TRANSACTION_COMPLETED, MANUAL_ADJUSTMENT)
- delta (DECIMAL 4,2)
- created_at (TIMESTAMPTZ)

Domain: property_core Schema (PostgreSQL — Metadata & Status Only)

property_listings table (PostgreSQL — authoritative state)

- id (UUID, PK)
- agent_id (UUID, FK → agent.agents.id)
- mongo_document_id (VARCHAR) — reference to full listing document in MongoDB
- listing_type (ENUM: RENTAL, SALE)
- city (ENUM: LAGOS, ABUJA, PORT_HARCOURT, OWERRI)
- property_type (ENUM: FLAT, DETACHED, SEMI_DETACHED, BUNGALOW, SELF_CONTAIN, ROOM)
- bedrooms (SMALLINT)

- annual_rent_kobo (BIGINT) — stored in kobo (1/100 of Naira) to avoid floating point
- verification_tier (ENUM: UNVERIFIED, BASIC_VERIFIED, FULLY_VERIFIED)
- listing_status (ENUM: DRAFT, PENDING_VERIFICATION, ACTIVE, SUSPENDED, RENTED, SOLD, ARCHIVED)
- geo_latitude (DECIMAL 10,8)
- geo_longitude (DECIMAL 11,8)
- created_at, updated_at, published_at (TIMESTAMPTZ)

MongoDB: property_listings collection (Full Listing Document) The full property document in MongoDB contains all the rich, schema-flexible attributes: property description text, full photo array (S3 URLs + metadata), floor plan URL, video tour URL, amenities map (generator: true, water: true, security: true, etc.), all fee structures (agency_fee_pct, legal_fee_pct, caution_months, service_charge), neighbourhood sub-document, AI-generated summary text, and the computed true_total_cost_kobo.

Domain: verification Schema (PostgreSQL)

verification_tasks table

- id (UUID, PK)
- listing_id (UUID, FK → property_core.property_listings.id)
- agent_id (UUID, FK)
- task_status (ENUM: QUEUED, PROCESSING, COMPLETED, FAILED, ESCALATED_HUMAN_REVIEW)
- document_type (ENUM: COO, DEED_OF_ASSIGNMENT, GOVERNORS_CONSENT, SURVEY_PLAN, AGENCY_MANDATE)
- s3_document_key (VARCHAR) — encrypted S3 object key
- ai_risk_score (DECIMAL 4,2) — 0.00 to 10.00
- ai_risk_flags (JSONB) — array of flag objects with type and description
- ai_raw_response_ref (VARCHAR) — reference to full AI response stored in S3 (not in DB for size)
- human_review_outcome (ENUM: NULL, APPROVED, REJECTED, INCONCLUSIVE)
- human_reviewer_id (UUID, NULLABLE, FK → user_kyc.users.id)
- human_review_notes (TEXT, NULLABLE)
- created_at, processing_started_at, completed_at (TIMESTAMPTZ)

fraud_reports table

- id (UUID, PK)
- reported_by_user_id (UUID, FK)
- reported_agent_id (UUID, FK, NULLABLE)

- reported_listing_id (UUID, FK, NULLABLE)
- report_type (ENUM: FAKE_LISTING, GHOST_PROPERTY, AGENT_IMPERSONATION, PRICE_BAIT_SWITCH, PAYMENT_BEFORE_VIEWING, OTHER)
- description (TEXT)
- evidence_s3_keys (TEXT[]) — array of S3 keys for uploaded evidence
- report_status (ENUM: SUBMITTED, UNDER_REVIEW, RESOLVED_FRAUD_CONFIRMED, RESOLVED_DISMISSED)
- resolution_notes (TEXT, NULLABLE)
- created_at, resolved_at (TIMESTAMPTZ)

3.2 Full ER Concept Diagram (Mermaid.js)

```
erDiagram
  USERS {
    uuid id PK
    varchar phone_number UK
    varchar email UK
    enum role
    enum account_status
  }

  KYC_PROFILES {
    uuid id PK
    uuid user_id FK
    varchar nin_hash
    varchar bvn_hash
    enum nin_verification_status
    enum bvn_verification_status
    varchar pii_vault_reference
  }

  AGENTS {
    uuid id PK
    uuid user_id FK
    varchar agency_name
    varchar cac_registration_number UK
    varchar lasrera_licence_number UK
    enum agent_type
    enum subscription_tier
    enum active_city
```

```
}

AGENT_TRUST_SCORES {
   uuid id PK
   uuid agent_id FK
   decimal composite_score
   decimal listing_accuracy_score
   decimal document_quality_score
   decimal review_sentiment_score
   decimal fraud_report_penalty
   int total_fraud_reports
   timestamptz last_calculated_at
}

PROPERTY_LISTINGS {
   uuid id PK
   uuid agent_id FK
   varchar mongo_document_id
   enum listing_type
   enum city
   enum property_type
   smallint bedrooms
   bigint annual_rent_kobo
   enum verification_tier
   enum listing_status
   decimal geo_latitude
   decimal geo_longitude
}

VERIFICATION_TASKS {
   uuid id PK
   uuid listing_id FK
   uuid agent_id FK
   enum task_status
   enum document_type
   varchar s3_document_key
   decimal ai_risk_score
   jsonb ai_risk_flags
   enum human_review_outcome
   uuid human_reviewer_id FK
```

```
}

FRAUD_REPORTS {
    uuid id PK
    uuid reported_by_user_id FK
    uuid reported_agent_id FK
    uuid reported_listing_id FK
    enum report_type
    enum report_status
    text description
}

VIEWING_APPOINTMENTS {
    uuid id PK
    uuid listing_id FK
    uuid renter_user_id FK
    uuid agent_id FK
    timestamptz scheduled_at
    enum status
    boolean agent_showed_up
    boolean listing_matched_photos
}

USERS ||--o| KYC_PROFILES : "has one"
USERS ||--o| AGENTS : "may be"
AGENTS ||--|| AGENT_TRUST_SCORES : "has one"
AGENTS ||--o{ PROPERTY_LISTINGS : "lists"
PROPERTY_LISTINGS ||--o{ VERIFICATION_TASKS : "has many"
PROPERTY_LISTINGS ||--o{ VIEWING_APPOINTMENTS : "has many"
USERS ||--o{ FRAUD_REPORTS : "submits"
AGENTS ||--o{ FRAUD_REPORTS : "may be subject of"
PROPERTY_LISTINGS ||--o{ FRAUD_REPORTS : "may be subject of"
```

3.3 Document Verification Storage Strategy

Document security is a critical compliance and trust requirement. The storage strategy has four layers:

Layer 1 — Client-Side Pre-Upload Before a document leaves the agent's device, it is assigned a client-generated UUID (the client document reference). This reference is included in the Kafka event payload, ensuring that document identity is established before network transmission.

Layer 2 — Encrypted Transport All document uploads use HTTPS/TLS 1.3 exclusively. Documents are uploaded directly to a pre-signed S3 URL generated by the backend — they do not pass through the application server. This reduces attack surface and offloads binary upload handling from the application tier.

Layer 3 — Server-Side Encryption at Rest All documents in S3 are encrypted at rest using AWS S3 SSE-KMS (Server-Side Encryption with AWS Key Management Service). Each city (Lagos, Abuja, PH, Owerri) has a separate KMS key, enabling data residency control and granular key rotation. The S3 bucket has: versioning enabled (documents cannot be silently overwritten), Object Lock with Compliance mode for verification-completed documents (immutability for audit), and no public access under any configuration.

Layer 4 — Access Control The S3 object key stored in the verification_tasks table is the only application-level reference to the document. Pre-signed URLs are generated on-demand, with a 15-minute expiry, and only for authorised system actors (the AI service, the human reviewer). No pre-signed URL is ever returned to an end user or an agent — they never have direct access to the document storage.

SECTION 4: INTEGRATION STRATEGY

4.1 External API Integration Architecture

BVN/NIN Validation (NIBSS / Identity Verification Providers)

Provider Strategy: Nigeria's BVN verification officially routes through NIBSS (Nigeria Inter-Bank Settlement System). However, NIBSS direct API access requires CBN licensing. NaijaNest will integrate via licensed identity verification middleware providers, Dojah, Smile Identity, or Youverify. all of which have valid NIBSS pass-through agreements and offer additional verification features (liveness detection, NIN slip verification).

Integration Pattern: The KYC Module calls the identity verification provider synchronously during user onboarding (the user waits for the result — acceptable because this is a one-time action). The integration uses the Circuit Breaker pattern (Resilience4j library) with fallback to a secondary provider if the primary is unavailable. The raw PII (NIN/BVN numbers) is never stored in the NaijaNest database. Only the verification status, a non-reversible hash of the identifier, and the provider's internal reference token are persisted.

Retry & Idempotency: The KYC service uses idempotency keys on all external calls. If a user's app crashes during verification, a retry uses the same key and does not create a duplicate verification request.

Google Maps / Geocoding API

Usage: Geocoding agent-submitted addresses to latitude/longitude coordinates, reverse geocoding for neighbourhood name resolution, distance calculation for "proximity to commute destination" queries, and static map image generation for listing detail pages.

Integration Pattern: All geocoding calls are cached in Redis with a 30-day TTL. The same address string should never result in a live API call more than once in 30 days. A nightly batch job geocodes any newly submitted listings that could not be geocoded in real-time (e.g., due to rate limits).

LLM Provider (OpenAI / Claude API)

Usage: The AI/ML Service uses LLM APIs for: natural language intent parsing from user search queries, generating AI summaries of property listings, analysing the semantic content of agent and landlord reviews, and powering the conversational chat interface.

Integration Pattern: The AI Service (Python) manages all LLM API calls. The Java monolith never calls the LLM directly. Communication is via internal API call (AI Service REST endpoint) for synchronous conversational responses, and via Kafka event for asynchronous tasks (listing summary generation, sentiment analysis batches).

Provider Resilience: The AI Service is configured with a primary provider (e.g., Anthropic Claude) and a fallback provider (e.g., OpenAI GPT-4o). If the primary provider exceeds latency thresholds or returns errors, the Resilience4j circuit breaker in the AI Service switches to the fallback. This is transparent to the Java monolith.

SMS & Push Notification Providers

SMS: Termii (Nigeria-first provider) as primary, Africa's Talking as fallback. Nigerian operators have known delivery rate variations. Termii's local routing infrastructure provides higher delivery rates for local numbers than international providers.

Push Notifications: Firebase Cloud Messaging (FCM) for both Android and iOS. The Notification Module in the monolith publishes to the notification.dispatch Kafka topic. A dedicated Notification Consumer service (can be extracted as a lightweight sidecar) consumes these events and routes to the appropriate channel (SMS for unregistered/low-tech users, Push for active app users, in-app notification for all).

4.2 AI Document Verification Engine — Java-to-ML Handoff Strategy

The handoff architecture between the Java backend and the Python AI service is designed around the Claim Check pattern combined with asynchronous event-driven processing.

Step 1 — File Upload & Claim Check The Java Verification Module receives the document upload request. It stores the file in S3 immediately and records the S3 object key (the "claim check") in the verification_tasks table. The original file binary never touches the Java application heap. This is critical for memory management, a 10MB PDF should not occupy JVM heap.

Step 2 — Event Publication The Verification Module publishes a verification.document.submitted Kafka event containing only: the verification task ID, the listing ID, the document type, and the S3 object key. No binary data in the event message.

Step 3 — AI Service Consumption The Python AI Service consumes the Kafka event. It generates a temporary pre-signed S3 URL (15-minute TTL) to retrieve the document. It runs the full forensic pipeline:

- PDF/image parsing and text extraction via OCR (Tesseract or Google Vision API)
- Image metadata EXIF analysis
- Font and format inconsistency detection
- Internal data cross-reference (e.g., does the property address in the survey plan match the C-of-O?)
- Reverse image search fingerprinting (checking for document reuse across listings)
- ML model scoring (a fine-tuned document authenticity classifier trained on known genuine and fraudulent Nigerian property documents — this is the long-term asset)

Step 4 — Result Return via Kafka The AI Service publishes a verification.task.completed event containing: task ID, risk score (0.0–10.0), structured flag array, OCR-extracted key data (property address, owner name, plot number), and a reference key to the full AI analysis report stored in S3 (for human reviewer access).

Step 5 — Java Backend Updates State The Verification Module's Kafka consumer processes the verification.task.completed event, updates the verification_tasks record, triggers trust score recalculation, and determines whether to route to auto-approval, auto-rejection, or human review based on the risk score thresholds defined in configurable business rules (stored in the application configuration, not hardcoded).

AI Service Handoff Architecture Diagram (Mermaid.js)

```
flowchart LR
  subgraph JAVA["Java Monolith"]
    VM[Verification Module]
    KC[Kafka Consumer - Result Handler]
  end

  subgraph STORAGE["Object Storage - S3"]
    RAW[raw-documents bucket - encrypted]
```

```
    RESULTS[ai-analysis-results bucket]
  end

  subgraph KAFKA["Kafka"]
    E1[verification.document.submitted]
    E2[verification.task.completed]
  end

  subgraph PYTHON["Python AI/ML Service"]
    CONS[Kafka Consumer]
    S3_FETCH[S3 Document Fetcher]
    OCR[OCR & Text Extraction]
    META[Metadata & EXIF Analyser]
    FONT[Font Inconsistency Detector]
    CROSS[Cross-Reference Engine]
    ML[ML Authenticity Classifier]
    SCORER[Risk Score Aggregator]
    RESULT_WRITER[Analysis Result Writer]
  end

  VM -->|Upload file| RAW
  VM -->|Publish event with S3 key| E1
  E1 -->|Consume| CONS
  CONS --> S3_FETCH
  S3_FETCH -->|Fetch via pre-signed URL 15min TTL| RAW
  S3_FETCH --> OCR
  S3_FETCH --> META
  OCR --> CROSS
  META --> FONT
  CROSS --> SCORER
  FONT --> SCORER
  ML --> SCORER
  SCORER --> RESULT_WRITER
  RESULT_WRITER -->|Store full report| RESULTS
  RESULT_WRITER -->|Publish result event| E2
  E2 -->|Consume| KC
  KC -->|Update verification_tasks| VM
```

SECTION 5: SECURITY & TRUST INFRASTRUCTURE

5.1 Security Architecture Overview

Security is not a feature layer in NaijaNest, it is woven into the infrastructure at every tier. The threat model is specific to the Nigerian context: sophisticated social engineering (fraudulent agents who are technically literate), data theft targeting KYC records (NIN/BVN data has significant black market value), account takeover attacks targeting diaspora users with high-value purchase intent, and insider threats (employees of partner KYC verification providers).

Authentication & Authorisation Architecture:

Authentication uses OAuth 2.0 / OpenID Connect with short-lived JWT access tokens (15-minute expiry) and long-lived refresh tokens (30-day expiry, stored in HttpOnly Secure cookies, never in mobile app local storage). The API Gateway validates JWT signatures on every request without a database roundtrip tokens are stateless and self-validating. Token revocation (for account suspension or logout) uses a Redis-based blacklist: the token JTI (unique token ID) is written to Redis with an expiry matching the token's remaining lifetime.

Authorisation uses Spring Security with method-level annotations enforcing Role-Based Access Control (RBAC). Roles are: RENTER, BUYER, AGENT_FREE, AGENT_STANDARD, AGENT_PREMIUM, TRUST_SAFETY_REVIEWER, ADMIN. A separate resource-level ownership check ensures that an agent can only modify their own listings even if their JWT is valid, they cannot access another agent's resource.

API Security:

Rate limiting is enforced at the API Gateway layer (Kong rate limiting plugin) on a per-user-ID and per-IP basis. Limits are tiered: unauthenticated endpoints (30 requests/minute per IP), authenticated standard endpoints (100 requests/minute per user), KYC/verification endpoints (5 requests/minute per user highly sensitive, low-volume operations). OWASP Top 10 mitigations are enforced at the gateway (SQL injection and XSS scanning via Kong's bot detection plugin) and at the application layer (parameterised queries exclusively via Spring Data JPA, output encoding via Jackson configuration).

All inter-service communication (Java monolith → AI Service, Java monolith → Kafka) uses mutual TLS (mTLS). No service-to-service communication occurs over unencrypted channels, even within the private VPC.

5.2 PII Handling Strategy for KYC Data

The guiding principle is data minimisation and tokenisation: the system should never store raw PII if a reference token or hash can serve the same functional purpose.

PII Vault Pattern:

Raw NIN and BVN numbers are processed as follows. On receipt of a KYC submission, the KYC Module immediately forwards the raw identifier to a dedicated, isolated PII Vault microservice (this is the one microservice that is extracted from Day 1, for isolation and compliance reasons). The PII Vault stores the raw identifier in an encrypted column (AES-256-GCM) in a separate PostgreSQL instance (not the main application database), accessible only by the PII Vault service itself, using a KMS-managed encryption key. The PII Vault returns a random, non-reversible vault reference token. This token is the only reference stored in the main kyc_profiles table.

No other service, team member, or log output ever contains the raw NIN or BVN. If a regulatory audit requires NIN/BVN access for a specific user, it routes through the PII Vault with a full audit log of who accessed it and why.

Data Residency:

All NaijaNest user data, including KYC records, is stored within AWS Africa (Cape Town) — the closest AWS region to Nigeria and the one most relevant for Nigerian data sovereignty discussions under the Nigeria Data Protection Act (NDPA) 2023. As AWS Nigeria (Lagos) availability becomes production-ready, a migration path to in-country hosting is part of the infrastructure roadmap.

NDPA 2023 Compliance:

The NDPA 2023 establishes rights for Nigerian data subjects analogous to GDPR. The KYC audit log (kyc_audit_log table) provides the foundation for: right of access (a user can request all data held about them), right to correction (a user can update incorrect KYC data, triggering a re-verification), and right to erasure (account closure triggers pseudonymisation of user records — real data is replaced with hashed references, preserving referential integrity for audit purposes without retaining PII).

5.3 Trust Score Service Design

The Trust Score is NaijaNest's core differentiation signal. It must be: accurate (reflecting genuine behavioural data, not gameable by agents), explainable (agents can understand why their score changed), real-time (updated within minutes of a trigger event), and auditable (every score change has a traceable cause).

Trust Score Composition Model

The composite Trust Score (0.0–10.0) is a weighted average of five sub-scores, each on a 0.0–10.0 scale:

Sub-Score 1 — Document Quality Score (Weight: 30%) Derived from the AI forensic analysis of all documents submitted by the agent. High-quality, low-risk documents increase this score. Flagged or rejected documents decrease it. An agent who has never submitted documents has a neutral baseline of 5.0 here.

Sub-Score 2 — Listing Accuracy Score (Weight: 25%) Derived from post-viewing feedback. After every completed viewing, renters are asked: "Did the property match the listing photos?" and "Was any information in the listing misleading?" Positive confirmations increase this score; negative reports decrease it with a weighted penalty that scales with the number of reports.

Sub-Score 3 — Transaction Completion Score (Weight: 20%) Ratio of confirmed viewings where the agent showed up on time versus no-shows or cancellations made with less than 2 hours notice. Ghost viewings (agent never responds after booking) carry the maximum penalty.

Sub-Score 4 — Review Sentiment Score (Weight: 15%) NLP sentiment analysis of textual reviews submitted by users. Not just star ratings — the AI reads the review text and scores for patterns of concern. A 3-star review that says "agent was fine but property had issues not mentioned" scores differently from a 3-star review that says "decent but just slow to respond." Patterns of similar negative themes across multiple reviews produce a compound penalty.

Sub-Score 5 — Community Trust Score (Weight: 10%) Based on platform longevity, total verified listings, and zero-fraud-report history. New agents start low here; established agents with a clean record build this over time. A single confirmed fraud report drops this component to 0.0, regardless of history.

Fraud Report Penalty (Additive Penalty, not a sub-score) Confirmed fraud reports apply an additive deduction to the composite score, on top of the weighted average. One confirmed report: -2.0 points. Two confirmed reports: -2.0 additional points (cumulative -4.0). Three confirmed reports: automatic suspension, score irrelevant.

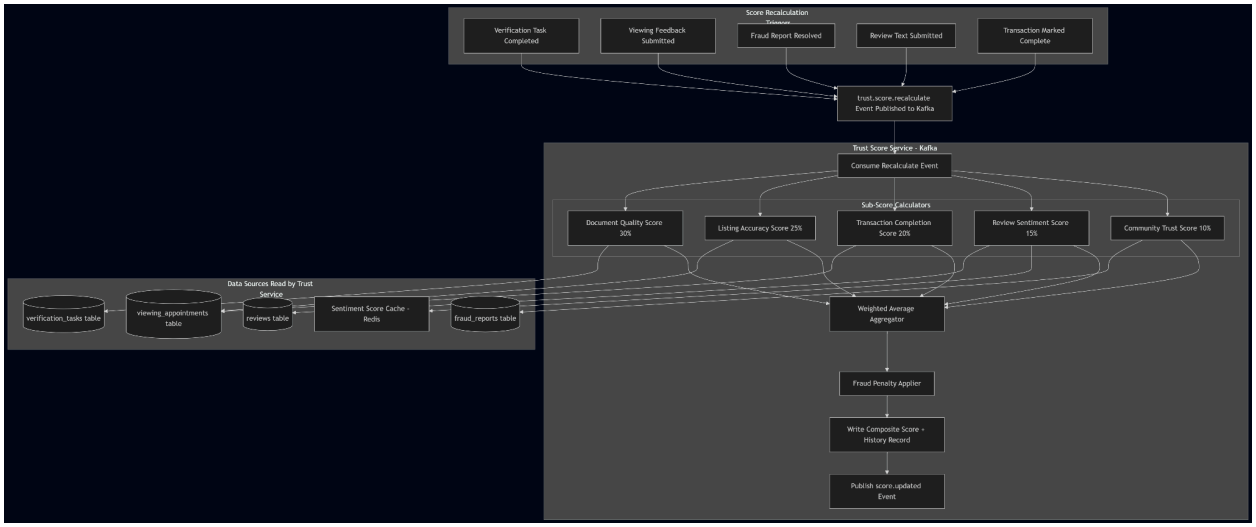
Trust Score Calculation Flow

The Trust Score service is a Kafka consumer that listens to trust.score.recalculate events. On receipt, it: queries the latest data for all 5 sub-score inputs (from their respective domain tables), computes each sub-score using the defined algorithm, applies the weighted average, applies any active fraud penalties, writes the new composite score to agent_trust_scores, and writes a score history record to agent_trust_score_history with the change trigger and delta. The entire calculation is synchronous within the consumer, the Trust Score is updated within seconds of the triggering event.

Trust Score Service Architecture Diagram (Mermaid.js)

```
flowchart TD
  subgraph TRIGGERS["Score Recalculation Triggers"]
    T1[Verification Task Completed]
    T2[Viewing Feedback Submitted]
    T3[Fraud Report Resolved]
    T4[Review Text Submitted]
    T5[Transaction Marked Complete]
  end

  KAFKA[trust.score.recalculate Event Published to Kafka]

  subgraph TRUST_SERVICE["Trust Score Service - Kafka Consumer"]
    CONSUME[Consume Recalculate Event]

    subgraph SUB_SCORES["Sub-Score Calculators"]
      DQ[Document Quality Score 30%]
      LA[Listing Accuracy Score 25%]
      TC[Transaction Completion Score 20%]
      RS[Review Sentiment Score 15%]
      CT[Community Trust Score 10%]
    end

    FP[Fraud Penalty Applier]
    WA[Weighted Average Aggregator]
    WRITE[Write Composite Score + History Record]
    NOTIFY[Publish score.updated Event]
  end

  subgraph DATA_SOURCES["Data Sources Read by Trust Service"]
    VT_DB[(verification_tasks table)]
    VA_DB[(viewing_appointments table)]
    FR_DB[(fraud_reports table)]
    REV_DB[(reviews table)]
    SENT_AI[Sentiment Score Cache - Redis]
  end

  T1 --> KAFKA
  T2 --> KAFKA
  T3 --> KAFKA
```

```
T4 --> KAFKA
T5 --> KAFKA

KAFKA --> CONSUME
CONSUME --> DQ
CONSUME --> LA
CONSUME --> TC
CONSUME --> RS
CONSUME --> CT

DQ --> VT_DB
LA --> VA_DB
TC --> VA_DB
RS --> REV_DB
RS --> SENT_AI
CT --> FR_DB

DQ --> WA
LA --> WA
TC --> WA
RS --> WA
CT --> WA
WA --> FP
FP --> WRITE
WRITE --> NOTIFY
```



5.4 Infrastructure Security: Network & Deployment

VPC Architecture: All NaijaNest infrastructure runs within a private Virtual Private Cloud. The only resources in the public subnet are the API Gateway/Load Balancer and the NAT Gateway. The application tier (Spring Boot instances), all databases, Kafka, Redis, and the AI Service are in private subnets with no direct internet access. Outbound traffic from private subnets routes through the NAT Gateway.

Database Security: PostgreSQL and MongoDB instances are deployed in private subnets with security group rules permitting inbound connections only from the application server security group on the specific database port. No database has a public endpoint. All database credentials are managed via AWS Secrets Manager with automatic 90-day rotation. The application retrieves credentials at startup from Secrets Manager credentials are never in environment variables, configuration files, or version control.

Secrets Management: All API keys (LLM providers, BVN/NIN verification, Google Maps, SMS) are stored in AWS Secrets Manager. The application uses the AWS SDK to retrieve secrets at runtime. Secret values are never logged, a custom Logback filter strips patterns matching known secret formats from all log output.

Dependency Security: The Gradle build pipeline includes OWASP Dependency Check as a mandatory build step. Any dependency with a known CVE above CVSS 7.0 fails the build. A dependency review workflow in the CI/CD pipeline (GitHub Actions) flags newly added transitive dependencies for manual security review.

Document Status: Architecture Draft v1.0 Prepared by: Backend Architecture & Lead Engineering Review